ChronoLens-SWE

Implementation spec for Google Antigravity

Meta Global AI Developer Hackathon · Theme 1: Build for Your World · Indie developer track

One-line pitch. ChronoLens-SWE watches a broken UI the way a human does, frame by frame. It measures what changed using pixel masks, reproduces the bug deterministically in a headless browser, fixes the code, and verifies the fix with the same measurement.

Key dates (from the official event page)

Milestone	Official time	In IST
Application deadline (or earlier if 20,000 spots fill)	Oct 25, 11:59 PM ET	Oct 26, 9:29 AM
Approval notification and kickoff livestream	Oct 26, by 9:00 AM ET	Oct 26, 6:30 PM
Live workshops and Discord office hours	Oct 27 to Nov 4	same dates
Project submissions due	Nov 4, 9:00 PM ET	Nov 5, 7:30 AM
Judging, then closing ceremony and winners	Nov 4 to 8, then Nov 9	same dates

Applications are scored equally on (1) how clear and feasible the idea is and how well it fits a theme, and (2) the technical skill shown in an AI project you describe. Indie-track projects are judged on creativity, technical ambition, and whether it works. The last point drives the scope decisions in this spec.

1. How to use this document with Antigravity

1. Create the repo **chronolens-swe**. Save this document as **docs/SPEC.md** (or attach the PDF) and paste Appendix A into the project's rules file or **AGENTS.md**.
2. Run **one Antigravity task per phase** from Section 6. Use planning mode and read the implementation plan before approving it. Reject steps that touch files outside the phase.
3. For every acceptance criterion, ask the agent to save evidence (test output, screenshots, a recording) under **runs/**.
4. Commit and tag after each phase (**phase-1**, **phase-2**, ...) so you can roll back when a later phase breaks something.

2. Product definition

	Definition
Problem	Transient UI bugs (races, flicker, z-index overlap, clipping) are hard to reproduce, and coding agents only see static code or single screenshots.
Inputs	Screen clip (MP4/MOV), spoken or typed description, repo path, app start command, base URL.
Outputs	Fix on a git branch, verification report (report.json and a before/after PNG card), PR description, and a Time-Travel Scrubber session.
Core loop	Ingest → Perceive → Detect → Reproduce → Root cause → Patch → Verify → Report (Section 4).
What the demo proves	On a benchmark of 3 seeded bugs, the system detects the anomaly frame, reproduces it, names the right file, fixes it, and passes 5 out of 5 verification runs.

3. Verified platform facts (Oct 23, 2026)

Every row below was read from the Meta docs on the date above. Build against these, not against the original pitch text.

Item	Verified fact
Endpoint and auth	OpenAI-SDK compatible. `base_url=https://api.meta.ai/v1`, key in env `MODEL_API_KEY`.
SAM 3.1 call	Responses API, model `sam-3.1`. Media goes in `input_image` or `input_video` (URL, base64 data URL, or uploaded `file_id`). Streaming is recommended.

Item	Verified fact
Concept prompts	One short noun phrase per request (for example "kanban card"). Comma lists return zero matches, "and" or a second text part segments only one concept, commands and questions are unreliable, negation/OR/spatial phrases are inconsistent. To segment several concepts, send one request each.
Video limits	MP4 or MOV with H.264, H.265 or MPEG-4. **WebM and VP9 are rejected.** Up to 15,000 frames per request. Up to 16 tracked objects per frame.
Output format	Special-token text, one line per frame, a box and mask per object. Read the frame marker on each line, never rely on line order. Use `pip install meta-sam-parser` (or `@meta-sam/parser` in TypeScript): `decode_mask_to_raster`, `decode_mask_to_rle`, `decode_mask_to_svg_path`.
Options	`metadata.mask_encoding="one_bit"` gives much smaller payloads. `metadata.include_confidence="true"` adds c (0 to 1) per record and needs meta-sam-parser 0.0.6 or later.
Box and point prompts	`build_object_prompt` (meta-sam-parser 0.1.0 or later) seeds up to 16 objects with ids per frame. Masks come back one_bit. `metadata.propagation_direction` is forward, backward or both.
Track identity	Ids are stable while an object stays in frame. Do not rely on them across entry, exit or full occlusion.
SAM price	$0.20 per 1,000 video frames. $2.50 per 1,000 images.
Muse Wonder	Model `muse-wonder-1.0`, 1M-token context, $4.00 input and $20.00 output per million tokens, cached input $0.20. Frontier coding and computer use. Wonder Full is available only through a Muse Code subscription.
Muse Glimmer	Open 30B model, Apache 2.0, runs on a single GPU, built for always-on local agents with tool use. Run it via Hugging Face, Ollama, LM Studio, Unsloth, or hosted on Fireworks, Together or OpenRouter.
Muse Spark 1.3, Voice Transcribe, Image	All three appear on Meta Model API. Read /docs/video-understanding, /docs/speech-to-text and /docs/image-generation before coding against them. Their details were not reviewed for this spec.

Claims in the original pitch that the docs do not support. Do not repeat them in the README, UI or demo.

- **"Object Multiplex"** does not appear in the SAM 3.1 docs I read. Leave the term out unless a Meta source confirms it.
- **One-pass multi-concept segmentation** is not possible. It is one concept per request, so fan out in parallel.
- **"Dozens of elements"** is capped at 16 tracked objects per frame.
- **"Mathematically prove"** overclaims. Say "quantitatively verify": IoU stability is a metric, not a proof.
- **Muse Glimmer does not launch browsers.** Playwright does. Glimmer is the local reasoning model that drives it.
- **"Anchored Composition"** (Muse Image) and **Meta Ray-Ban Display viewport specs** are unverified. Treat both as optional.

4. Architecture

```
clip.mp4 + voice note or typed text
       |
[A] INGEST ....... ffprobe + ffmpeg -> H.264 MP4 (faststart, constant fps); fps, frame count, size
       |
[B] PERCEIVE ..... Muse Voice Transcribe -> transcript
       |           Muse Spark 1.3        -> concepts[<=5 noun phrases], interaction, suspect window
       |           SAM 3.1 x N concepts (parallel, streaming, one_bit, confidence) -> tracks.json
       |
[C] DETECT ....... per-track, per-frame metrics -> anomalies[] (frame, t_ms, type, severity)
       |
[D] REPRODUCE .... Muse Wonder writes a Playwright script; run with video + trace + HAR + DOM probe
       |           + sync flash; re-run B and C on the repro clip; match the anomaly signature
       |
[E] ROOT CAUSE ... Muse Wonder (1M ctx): repo + probe log + HAR slice + console + anomaly signature
       |
[F] PATCH ........ Muse Wonder edits in a git worktree; run tests, lint, typecheck
       |
[G] VERIFY ....... same script x5 on the patched build; B and C again; pass/fail gate
       |           fail -> back to [E] with failure evidence (max 3 loops)
[H] REPORT ....... before/after card (HTML -> PNG), PR markdown, report.json, scrubber session
```

Design principle: hybrid truth

During reproduction runs the DOM probe gives exact boxes, z-index and event timing, while SAM reports what pixels actually showed. A disagreement between the two (DOM says visible, pixels say covered) is itself a bug signal. SAM is essential for the **user-supplied clip**, where no DOM data exists, and for visual verification. In repro runs, DOM boxes from frame 0 can seed SAM box prompts (`build_object_prompt`) to track exact elements by id.

5. Stack and repository layout

- **Backend:** Python 3.11, FastAPI (SSE for live run events), `openai` SDK, `meta-sam-parser`, numpy, opencv-python, Playwright, ffmpeg.
- **Web:** Vite + React + Tailwind. Run view plus the Time-Travel Scrubber.
- **Models at runtime:** sam-3.1, muse-wonder-1.0, Muse Spark 1.3, Muse Voice Transcribe. Glimmer only for the stretch feature.
- **Build tool:** Antigravity writes the code. The finished product calls Meta models only through one wrapper.

```
chronolens-swe/
  AGENTS.md                       # rules (Appendix A)
  docs/SPEC.md
  backend/app/main.py             # FastAPI: POST /runs, GET /runs/{id}, GET /runs/{id}/events
  backend/app/clients/meta.py     # OpenAI SDK wrapper: retries, cost meter, the only place that calls Meta
  backend/app/pipeline/           # ingest, perceive, metrics, reproduce, rootcause, patch, verify, report
  backend/app/probe/chronolens-probe.js
  backend/tests/
  web/                            # run view + Time-Travel Scrubber
  demo-app/                       # Kanban app with 3 seeded bugs (?bug=1,2,3)
  bench/                          # seeded-bug benchmark runner and results.md
  runs/                           # per-run artifacts (gitignored)
```

6. Phases, deliverables and acceptance criteria

Phase	Window	Deliverable	Acceptance criteria
P0	Oct 23 to 25	Submit the application. Scaffold the repo. Try SAM 3.1 in the Meta playground on one screen clip.	Application submitted before the deadline. Noun phrases that segment UI elements well are written down.
P1	Days 1-2	demo-app with 3 seeded bugs, record pipeline.	`make record BUG=1` yields an H.264 MP4 that sam-3.1 accepts. Bugs are deterministic (seeded latency). Each bug writes a ground-truth marker to the probe log.
P2	Days 2-3	SAM fan-out, parser, metrics, detector.	Anomaly found within ±3 frames of ground truth on all seeded bugs. Zero anomalies on healthy baseline clips. Prompts and thresholds live in one config file.

Phase	Window	Deliverable	Acceptance criteria
P3	Days 4-5	DOM probe, reproduce loop, root cause.	Signature match on at least 2 of 3 bugs within 3 attempts. Root cause names the correct file for at least 2 of 3.
P4	Days 5-6	Patch, verify, report.	Patched build passes 5 of 5 runs for at least 2 of 3 bugs with no new anomaly types. Before/after card and PR markdown generated.
P5	Days 7-8	Time-Travel Scrubber UI.	Scrubbing the timeline overlays masks and shows DOM events, network calls and the code lines for that frame.
P6	Day 9	**One** stretch feature: Chaos Ghost Swarm or Responsive Auto-Patcher.	Ship one narrow working version. Cut the other without guilt.
P7	Day 10	Freeze, README, demo video, benchmark table, submit.	Repo public, clean-clone run works, submitted well before Nov 4, 9:00 PM ET.

Demo app: three seeded bugs (ground truth)

Bug	Symptom	Seeded cause
1 Race	Card snaps back and the column flickers after a fast drag.	Optimistic move, then a slower stale server response overwrites state. No request versioning in the move reducer.
2 Z-index	Modal sits under the sticky navbar below ~900 px width.	Modal rendered inside a transformed container, which creates a new stacking context.
3 Clipping	Button label is cut off at 360 px width.	Fixed-width button with `white-space: nowrap`.

Keep each bug's reference fix in `bench/fixes/bugN.patch` and never include it in any model's context. It exists only to grade the system.

7. Detection and verification metrics

Notation: concept c, track k, frame t, mask M, confidence conf. Drop records with conf < 0.5. All thresholds live in config and are calibrated on healthy baseline clips of the same scripted interaction.

Signal	Definition	Flags
Dropout	Track seen at frames t1 and t2 > t1+1 but absent in between.	Flicker, unmount/remount
Instance flicker	Instance count of a concept changes then reverts within 6 frames.	Duplicate or ghost render
Teleport	Centroid jump > max(3 × baseline p99 per-frame displacement, 40 px).	Snap-back, layout jump
Shape stability	IoU of M_t with M_{t-1} after translating M_{t-1} by the centroid shift. Below 0.85 is a deformation.	Partial render, reflow
Occlusion ratio	Visible mask area ÷ DOM box area (repro runs only). Below 0.9 while the DOM says visible.	z-index, overflow
Cross-concept overlap	IoU(M_a, M_b) > 0.05 for pairs marked must-not-overlap (modal vs navbar).	Overlap bugs
Clipping	Mask touches the viewport edge, or DOM scrollWidth > clientWidth.	Text/button clipping

Signature = sorted set of (anomaly type, concept) inside a ±300 ms window. Reproduction succeeds when the repro clip's signature matches the user clip's. **Verify passes** when the signature is absent in 5 of 5 runs, median shape-stability IoU ≥ 0.95 on tracked concepts, no new anomaly types appear, and a Playwright functional assertion passes. Report this as a *stability score*, not a proof.

8. DOM probe, time sync and the Scrubber

- **Probe:** inject `chronolens-probe.js` with `page.add_init_script`. On every `requestAnimationFrame` log `performance.now()` plus id, `getBoundingClientRect`, z-index, opacity, visibility and display for elements matching `[data-cl]` and configured selectors. Add a MutationObserver log with timestamps.

- **Network:** Playwright HAR plus request/response event timestamps. Console output captured with timestamps.
- **Sync flash:** in test runs only, paint an 8×8 magenta square for exactly one frame after load. Detect the first magenta frame in the video to compute the offset between the probe clock and the video frame index.
- **Video format:** Playwright records WebM, which SAM rejects. Convert with `ffmpeg -i in.webm -c:v libx264 -pix_fmt yuv420p -movflags +faststart -fps_mode cfr -r 30 out.mp4` (older ffmpeg: `-vsync cfr`). Assert H.264 with ffprobe before every upload.
- **Scrubber:** an HTML5 video with a canvas overlay. Backend converts masks to SVG paths (`decode_mask_to_svg_path`), simplified and stored per frame as JSON. Frame index = round(currentTime × fps), using `requestVideoFrameCallback` where available. Side panel shows DOM events, HAR entries and the evidence lines from the root-cause JSON for that frame. Alternative: the official `@meta-sam/react` `Video` component, which overlays masks frame-accurately.

9. Model prompts and schemas

Root-cause system prompt (Muse Wonder)

You are the root-cause engine of ChronoLens-SWE. You receive (1) an anomaly signature with frame indexes and timestamps, (2) the DOM probe log for ±300 ms around it, (3) the HAR slice, (4) console output, (5) source files. Rules: cite at least two pieces of evidence from the telemetry, never from guesswork. Name the exact file and line range. Propose one falsifiable check a script could run. If the evidence is insufficient, say so and request one specific extra probe. Output only the JSON schema.

```
{
  "hypothesis": "string",
  "files": [{"path": "string", "lines": "start-end", "why": "string"}],
  "evidence": [{"source": "probe|har|console|mask-metric|code", "ref": "string", "t_ms": 0}],
  "falsifiable_check": "string",
  "confidence": 0.0
}
```

Patch prompt (Muse Wonder)

Make the smallest change that removes the cause. Do not refactor. Do not edit tests to make them pass. Keep public APIs. Output a unified diff and a one-paragraph rationale.

Concept proposal (Muse Spark 1.3)

Ask for at most 5 short noun phrases naming concrete visible UI objects ("kanban card", "modal dialog", "loading spinner", "navigation bar"), plus the interaction and suspect time window. Validate each phrase in code: no commas, no verbs, 1 to 4 words. Then send one SAM request per phrase.

10. Cost budget

SAM cost per clip and concept = frames ÷ 1000 × $0.20. A 10 s clip at 30 fps is 300 frames, so $0.06. A full bug run uses 7 clips (user, repro, 5 verify) × 4 concepts = 28 concept-clip runs ≈ $1.68, or about $3.36 at 60 fps. Muse Wonder is dominated by input tokens: 200k input tokens ≈ $0.80, and cached input is $0.20 per million, so repeated loops over the same repo are cheap. A full loop with two iterations is likely in the single-digit dollars (estimate). Build a cost meter into `clients/meta.py` and show it in the UI. Check pricing-rate-limits and whether the hackathon provides credits.

11. Risks and mitigations

Risk	Mitigation
SAM mis-segments UI elements (trained for natural imagery, zero-shot here)	Test noun phrases in the playground first (P0). Prefer specific phrases. In repro runs, seed box prompts from DOM boxes. Report per-concept confidence.
More than 16 objects on screen	Track only the 16 most relevant elements. Say so in the README.
WebM from Playwright is rejected	Mandatory ffmpeg step plus an ffprobe assertion.

Risk	Mitigation
Race conditions are nondeterministic	Seeded latency in the demo app, 5 verification runs, signature-based comparison.
Scope creep in 10 days	Freeze features on Day 9. Stretch features are optional. Core loop plus Scrubber is the product.
Overclaiming loses credibility	Use the wording in Section 3. Include a Limitations section with the benchmark.
Secrets leak into model context	Redact .env and key-like strings before building repo context. Keys only from environment.
Unsafe auto-fixes	Patches only in a git worktree on a new branch. Never touch main. Never auto-merge.
Glimmer needs a GPU	Use a hosted Glimmer endpoint (Fireworks, Together, OpenRouter) or skip the stretch feature.

12. Stretch features (pick one in P6)

- **Chaos Ghost Swarm:** Muse Glimmer drives Playwright with tool calls (click, drag, resize, network throttling through CDP). Run SAM only when the DOM probe flags a suspicious state, to save cost. Flagged clips enter the main pipeline.
- **Responsive Auto-Patcher:** run the pipeline at three viewports (for example 1440×900, 768×1024, 390×844) and flag clipping and overlap. Add a glasses viewport only if Meta publishes its display specs.

13. Demo script (2:00) and submission checklist

Time	Scene
0:00-0:15	Real flicker clip. State the problem in one sentence.
0:15-0:45	Upload clip and voice note. SAM masks fill the timeline. The anomaly frame lights up.
0:45-1:15	Reproduction in headless browser. Root-cause JSON with evidence lines.
1:15-1:40	Patch, 5 out of 5 verification runs, before/after card.
1:40-2:00	Scrubber across the timeline, then the benchmark table (X of 3 seeded bugs fixed).

- Public GitHub repo, README with architecture diagram and a table of which Meta model does what.
- Run instructions that work from a clean clone. No keys committed.
- 2-minute demo video, benchmark results, and an honest Limitations section.
- Submit hours early. The deadline is Nov 4, 9:00 PM ET (Nov 5, 7:30 AM IST).

Appendix A. Antigravity rules block

```
PROJECT RULES: ChronoLens-SWE
1. Python 3.11 with type hints; ruff for lint; pytest for metrics and pipeline tests.
2. Never hard-code keys. Read MODEL_API_KEY from the environment. .env stays in .gitignore.
3. All Meta API calls go through backend/app/clients/meta.py (retries, cost meter, logging).
4. SAM: exactly one concept per request. Never combine concepts in one prompt.
5. Videos sent to SAM must be H.264 MP4. Assert with ffprobe before upload.
6. Key SAM results by the frame marker in each line, never by line order.
7. Use meta-sam-parser for parsing. Request one_bit masks and include_confidence.
8. Patching happens in a git worktree on branch chronolens/fix-<run-id>. Never touch main.
   Never auto-merge. Never edit tests to make them pass.
9. Redact .env and secret-like strings before sending repo context to any model.
10. Each phase ends with: tests green, evidence saved under runs/, commit tagged phase-N.
11. Do not claim capabilities the Meta docs do not support (see Section 3 callout).
12. Prefer the simplest implementation that meets the acceptance criteria. Ask before adding scope.
```

Appendix B. Reference links

- Event: dev.meta.ai/events/global-hackathon
- SAM: dev.meta.ai/docs/media-segmentation · /docs/sam/segmenting · /docs/sam/reading-segmentation · /docs/sam/client-libraries
- Models: dev.meta.ai/models/muse-wonder/ · /models/muse-glimmer/ · /models/sam-3-1/
- Capabilities: dev.meta.ai/docs/video-understanding · /docs/speech-to-text · /docs/image-generation · /docs/pricing-rate-limits
- Agents: dev.meta.ai/docs/computer-use · /docs/cookbook/computer-use · /docs/cookbook/agent-patterns